Google Cloud Platform + AI Fundamentals

Universidad Austral
Facultad de Ingeniería
Primer Cuatrimestre 2026

Profesores
Mora Villa Abrille (BVilla@austral.edu.ar)
Rodrigo Pazos (RPazos@austral.edu.ar)

Compute Engine

Run large-scale workloads on virtual machines hosted on Google's infrastructure

App Engine

A platform for building scalable web apps and mobile backends

Container Engine

Run Docker containers on Google's infrastructure, powered by Kubernetes

Cloud Run

Fully managed compute platform for deploying and scaling containerized applications quickly and securely.

Cloud Functions

A serverless platform for building event-based microservices triggered by events in GCP

La VMs y los MIGs toman tiempo

- Cuando creamos una VM en casi todos los casos el **provisioning** se hace **“on-best effort”**. Configurar y encender la VM también toma tiempo
- **Un MIG responde a subas y bajas de demanda con una ventana de por lo menos 10 minutos** entre lo que se conoce como el ***initialization*** *process* y el ***stabilization*** *process*

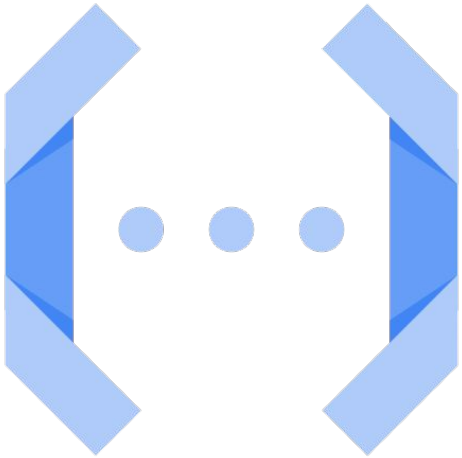
La VMs y los MIGs toman tiempo

- Todo este tiempo asociado a la toma de decisiones **consume recursos** y **GCP nos cobra por esos recursos**
- Pensando el caso extremo: **qué pasa cuando tengo un servicio que la mayor parte del tiempo responde una request cada 10 minutos?**
- **Termino teniendo un MIG o un servidor dedicado para no hacer nada la mayor parte del tiempo**

Serverless

- En 2006 una empresa llamada Zimki crea la primera solución serverless donde los desarrolladores entregaban su código de Javascript y la plataforma de encargaba de la infraestructura y el escalado
- GCP presenta en 2008 App Engine que permite correr aplicaciones (con limitaciones en lenguaje y acceso a funciones del OS)
- AWS presenta 2014 el primer servicio FaaS con AWS Lambda, que permitía correr fragmentos de código en respuesta a eventos y solo se cobraba por el tiempo de ejecución

Cloud Run Functions



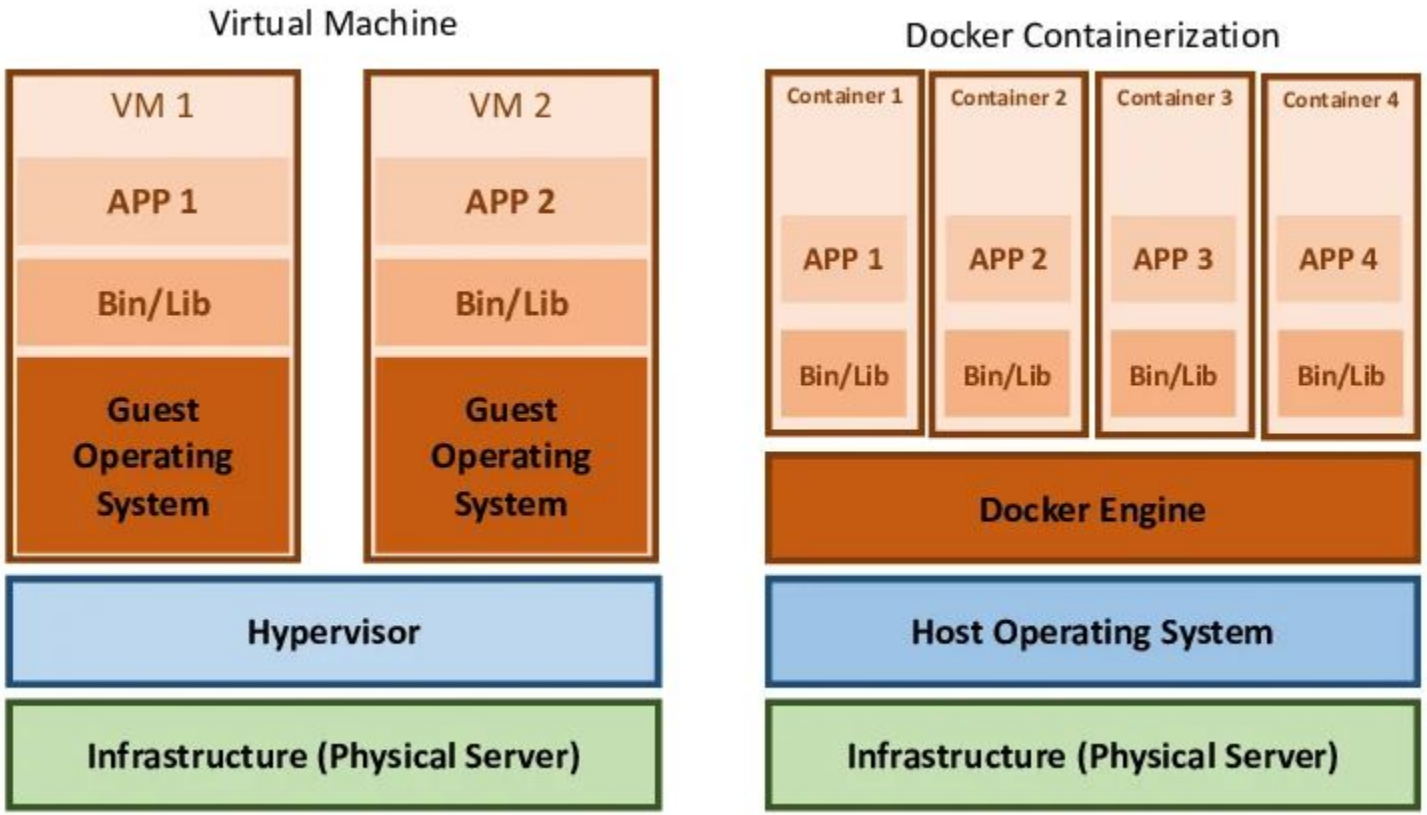
- Una solución serverless para correr pequeños fragmentos de código ante eventos concretos
- Funcionan con Functions Framework, un SDK disponible para algunos lenguajes y nos pide cierta estructura (ej: un main.py y un requirements.txt en Python)
- La primera generación era más limitada (límites de vCPU, RAM y timeout de ejecución)

	Cloud Run functions	Cloud Run functions (1st gen)
Request timeout	• Up to 60 minutes for HTTP-triggered functions • Up to 9 minutes for event-driven functions created with the Cloud Functions v2 API	• Up to 9 minutes
Instance size	Up to 16 GiB RAM with 4 vCPU	Up to 8 GB RAM with 2 vCPU
Concurrency	Up to 1000 concurrent requests per function instance	1 concurrent request per function instance
Traffic splitting	Supported	Not supported

El contexto es importante

- Cómo construimos una aplicación más compleja sobre serverless? Ejecutar fragmentos de código puede funcionar para pequeñas soluciones pero no para aplicaciones complejas.
- Qué es lo que no manejamos en los primeros serverless? El ambiente de ejecución: archivos de configuración, dependencias del sistema operativo, etc.
- En las VMs eso está disponible pero aca como le damos un artefacto a GCP para que administre esto?

APM TERMINALS
MAERSK LINE
MARSEILLE MÆRSK
KØBENHAVN



Ver sección 3.2.2 de Distributed Systems de Tannenbaum

El contexto es importante

- Puedo publicar y compartir imágenes (artefactos) que contienen la configuración del sistema operativo, las dependencias y el contexto necesario disponible
- GCP crea contenedores en una fracción del tiempo porque se ahorra los pasos de provisioning y la inicialización es mucho menos compleja
- Obviamente no todas las imágenes son iguales y GCP tiene mejores prácticas

Cloud Run Service

- Puedo publicar y compartir imágenes (artefactos) que contienen la configuración del sistema operativo, las dependencias y el contexto necesario disponible
- GCP crea contenedores en una fracción del tiempo porque se ahorra los pasos de provisioning y la inicialización es mucho menos compleja
- Obviamente no todas las imágenes son iguales y GCP tiene mejores prácticas

Request-based billing

Iowa (us-central1)

Mostrar opciones de descuento

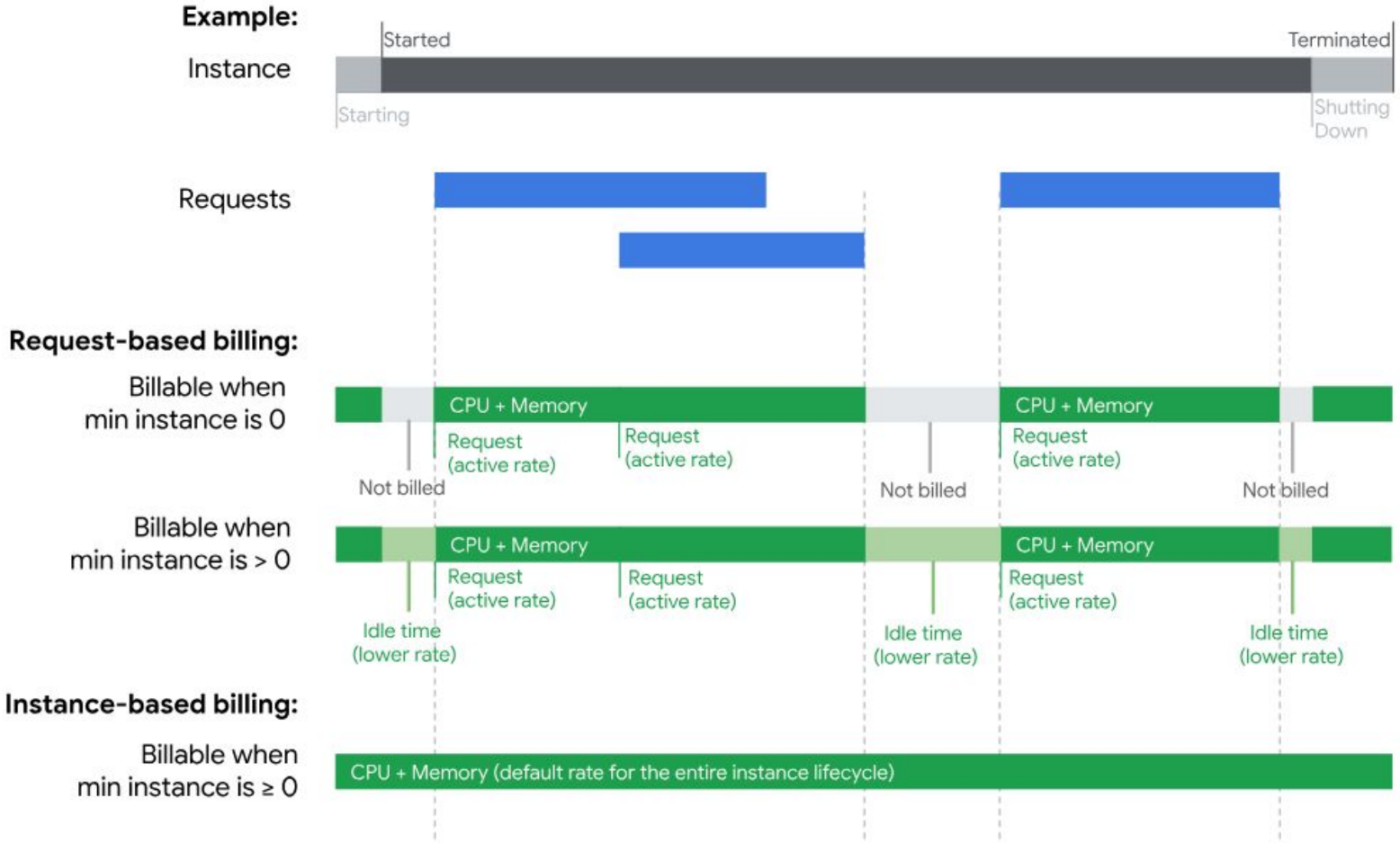
Recurso	Tipo	Predeterminado (USD)
CPU (por segundo de CPU virtual)	Tiempo de actividad	USD 0.000024
	Tiempo de inactividad (instancia mínima[1])	USD 0.0000025
Memoria (por GiB-segundo)	Tiempo de actividad	USD 0.0000025
	Tiempo de inactividad (instancia mínima[1])	USD 0.0000025
Solicitudes (por 1,000,000)	No corresponde	USD 0.40

Instance-based billing

Iowa (us-central1)

Mostrar opciones de descuento

Recurso	Predeterminado (USD)
CPU (por segundo de CPU virtual)	USD 0.000018
Memoria (por GiB-segundo)	USD 0.000002
Tipo de GPU NVIDIA-L4 Sin redundancia zonal (por segundo)	USD 0.0001867
Tipo de GPU NVIDIA-L4 Redundancia zonal (por segundo)	USD 0.0002909
Tipo de GPU NVIDIA RTX Pro 6000 Sin redundancia zonal (por segundo)	USD 0.00036522
Tipo de GPU NVIDIA RTX Pro 6000 Redundancia zonal (por segundo)	USD 0.00056913


Example:
Instance
Started
Terminated
Starting
Shutting Down
Requests
Request-based billing:
Billable when min instance is 0
CPU + Memory
Request (active rate)
Request (active rate)
Not billed
Not billed
CPU + Memory
Request (active rate)
Not billed
Billable when min instance is > 0
CPU + Memory
Request (active rate)
Request (active rate)
Idle time (lower rate)
Idle time (lower rate)
CPU + Memory
Request (active rate)
Idle time (lower rate)
Instance-based billing:
Billable when min instance is ≥ 0
CPU + Memory (default rate for the entire instance lifecycle)

Cloud Run Service

- Puedo publicar y compartir imágenes (artefactos) que contienen la configuración del sistema operativo, las dependencias y el contexto necesario disponible
- GCP crea contenedores en una fracción del tiempo porque se ahorra los pasos de provisioning y la inicialización es mucho menos compleja
- Obviamente no todas las imágenes son iguales y GCP tiene mejores prácticas